

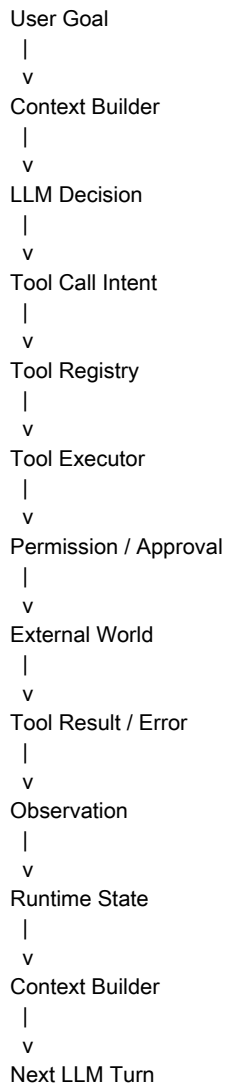
# Day05 Part H 学习文档 v1.0 : Multi Tool Loop ( Agent Execution Loop )

本文是《从零实现 Agent Runtime》学习阶段的 Day05 Part H 正式学习文档。

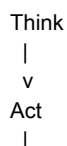
Day05 Part A 建立 Tool Calling 基础模型，Part B 解释 LLM 如何做 Tool Decision，Part C 说明 Tool Schema 如何定义行动契约，Part D 说明 Tool Registry 如何管理能力空间，Part E 说明 Tool Executor 如何把 Tool Call Intent 转成真实执行，Part F 引入 Permission 与 Human Approval 治理层，Part G 说明 Tool Result 如何转成 Observation 回流 Runtime。Part H 开始回答执行系统的收束问题：Agent 如何连续调用多个 Tool，并在反馈中决定继续、停止或切换行动？

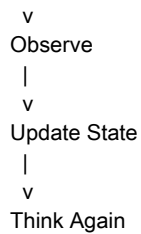
## 本节定位

Part A-G 已经完成一次 Tool 生命周期：



Part H 关注的是这条链路如何形成多步循环：





本节核心结论是：

单次 Tool Calling 只能让模型做一次动作；Multi Tool Loop 才让 Agent 在反馈中持续完成目标。Runtime 需要维护 Tool Call Chain、Observation、Runtime State、Loop Control 和 Stop Condition，才能把一次函数调用升级成可治理的 Agent Execution Loop。

## 目录

为什么 Agent Loop 不等于一次 Tool Call

Multi Tool Loop 的本质

Multi Tool Loop 在 Runtime 中的位置

Runtime State 与 Step

Tool Call Chain

ToolCallRecord

Tool Call 与 Observation 的关系

Agent Loop 作为 State Machine

Stop Condition

Loop Control

Retry 与 Loop 的区别

Loop Guard

Agent Executor Loop

Agent Executor 与 Tool Executor

Step Runner

Tool Call Queue

Sequential Tool Execution

Parallel Tool Execution

Tool Dependency Graph

OpenAI Agents SDK / Claude Code / LangGraph 映射

Multi Tool Loop 与 Workflow

Open-ended Capability 与超级 App 类比

Agent as Interface 与 Agent as Worker

AI Assistant + Workflow

AI Assistant + Workflow 的开源方案

mini-agent-runtime 设计

工业术语映射

面试视角

下一节学习计划

写书 TODO

写书素材

本 Part 核心认知升级

工业级实现

知识地图

思考题

前置问题回收

## 为什么 Agent Loop 不等于一次 Tool Call

单次 Tool Calling 的流程是：

```
graph TD
    User --> LLM
    LLM --> ToolCall[Tool Call]
    ToolCall --> ToolResult[Tool Result]
    ToolResult --> Answer
```

例如：

User:  
查询订单状态

LLM:  
调用 get\_order

Tool:  
返回订单状态

Agent:  
回答用户

这只是 Single Tool Execution。

真实任务通常不止一个动作。

例如用户说：

帮我查一下订单，如果已经送达，给客户发一个满意度调查。

这里至少需要：

查询订单

根据订单状态判断是否送达

如果已送达，发送满意度调查

根据发送结果生成最终回答

完整过程是：

```
Think
|
v
get_order()
|
v
Observation: 订单已送达
|
v
Think
|
v
send_survey()
|
v
Observation: 调查已发送
|
v
Think
|
v
Final Answer
```

这就是 Multi Tool Loop。

## Multi Tool Loop 的本质

Tool Calling 让 LLM 获得行动能力。

但严格来说：

只有 Tool Calling + Loop，才让系统接近真正的 Agent。

单次 Tool 更像：

```
Input
|
v
LLM
|
v
Action
|
v
Output
```

Agent Loop 则是：

```
Goal
|
v
Reason
|
v
Action
|
v
Observation
|
v
```

```
Update State
|
v
Reason Again
```

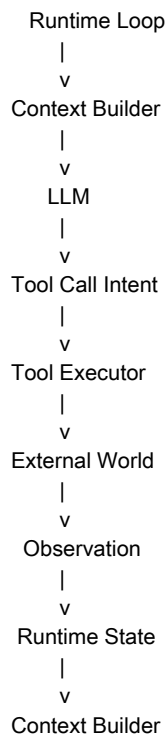
核心差异是：

```
Iteration
```

也就是 Agent 并不提前知道所有步骤，而是在环境反馈中动态决定下一步。

## Multi Tool Loop 在 Runtime 中的位置

从 Runtime 视角看，Multi Tool Loop 是 Day05 执行系统的收束：



这里形成一个环。

工业术语中常见表达包括：

- Agent Loop
- ReAct Loop
- Agent Execution Loop
- Reasoning-Acting Loop
- Control Loop

## Runtime State 与 Step

Multi Tool Loop 不能只靠：

```
await tool.execute()
```

Runtime 至少需要记录：

```
interface RuntimeState {
    goal: string;
```

```

messages: Message[];
toolCalls: ToolCallRecord[];
observations: Observation[];
step: number;
status:
  | "thinking"
  | "executing"
  | "waiting"
  | "completed"
  | "failed";
}

```

每一轮推进都对应一个 Step。

例如初始状态：

```

{
  "step": 0,
  "status": "thinking"
}

```

LLM 决定调用 Tool：

```

{
  "tool": "get_order"
}

```

Runtime 更新状态：

```

{
  "step": 1,
  "status": "executing"
}

```

Tool 执行完成后写入 Observation：

```

{
  "step": 2,
  "status": "thinking",
  "observations": [
    {
      "type": "tool_result",
      "summary": "订单已发货"
    }
  ]
}

```

下一轮 LLM 看到 Goal 和 Observation 后，再决定是否调用下一个 Tool。

## Tool Call Chain

Tool Call Chain 指一个任务中多个 Tool Call 之间的执行轨迹。

例如：

```

User Goal
|
v
Tool Call 1: get_order
|
v
Observation 1: ORDER_SHIPPED
|
v
Tool Call 2: get_delivery_info
|
v

```

Observation 2: DELIVERY\_DATE  
|  
v  
Final Answer

Runtime 需要记录 Tool Call Chain，因为它要回答：

当前执行到哪一步？

为什么调用这个 Tool？

如果失败，如何恢复或停止？

如何 Debug、Replay、Trace 和 Evaluation？

如果没有 Chain，只保存最后结果，Runtime 会丢失关键因果关系。

## ToolCallRecord

最小结构可以是：

```
interface ToolCallRecord {  
  id: string;  
  toolName: string;  
  arguments: unknown;  
  status:  
    | "pending"  
    | "running"  
    | "success"  
    | "failed";  
  startedAt: number;  
  completedAt?: number;  
  observationId?: string;  
}
```

示例：

```
{  
  "id": "call_001",  
  "toolName": "get_order",  
  "arguments": {  
    "orderId": "123"  
  },  
  "status": "success",  
  "observationId": "obs_001"  
}
```

这里要注意：

ToolCall 和 Observation 是两个对象。

ToolCall 表示：

我要做什么

Observation 表示：

世界发生了什么

## Tool Call 与 Observation 的关系

错误设计是：

Tool Call  
|

v  
Tool Result

然后结束。

Agent Runtime 应该是：

ToolCall  
|  
v  
Execution  
|  
v  
Observation

三者含义不同：

| 对象 | 含义 | | --- | | ToolCall | Action Intent | | ToolResult | Execution Output | | Observation | Runtime Feedback |

Observation 通常需要关联 Tool Call：

```
interface Observation {  
  id: string;  
  type: "tool_result" | "tool_error";  
  toolCallId: string;  
  summary: string;  
  payload?: unknown;  
}
```

这样 Runtime 才知道哪个 Tool 出错、哪个 Tool 成功，以及后续决策基于哪次执行反馈。

## Agent Loop 作为 State Machine

Part G 说明：

Observation  
|  
v  
State Transition

Part H 进一步说明：

State Transition  
|  
v  
Loop

所以 Agent Runtime 越来越像 State Machine。

典型状态：

IDLE  
|  
v  
THINKING  
|  
v  
EXECUTING\_TOOL  
|  
v  
OBSERVING  
|  
v  
THINKING  
|



v  
COMPLETED

事件包括：

ToolCalled  
ToolCompleted  
ToolFailed  
ApprovalResolved  
AgentStopped  
AgentCompleted

Reducer 根据事件更新 State。

这也是为什么 Agent Runtime 不应该只是：

callLLM()

而更像：

Event-driven Runtime

## Stop Condition

Multi Tool Loop 最大的工程风险是无限循环。

例如：

LLM: 调用 search\_tool  
Tool: 没有结果  
LLM: 再次调用 search\_tool  
Tool: 没有结果  
LLM: 再次调用 search\_tool  
...

因此停止条件必须由 Runtime 控制，而不是完全交给 LLM。

常见 Stop Condition：

Final Answer Stop

Max Step Limit

Timeout Stop

Token Budget Stop

Cost Budget Stop

Tool Failure Stop

Human Stop / Cancellation

## Final Answer Stop

最自然的停止条件是 LLM 输出 Final Answer：

```
{  
  "type": "final",  
  "content": "订单已经发货，预计明天送达。"  
}
```

Runtime 将状态更新为：

completed

## Max Step Limit

生产系统必须限制最大步骤数：

```
const maxSteps = 20;

if (state.step >= maxSteps) {
  return {
    status: "failed",
    reason: "max steps exceeded"
  };
}
```

Agent Loop 理论上可以无限执行，但生产系统必须有限。

## Timeout Stop

Runtime 需要限制总运行时间：

```
interface RuntimeConfig {
  timeoutMs: number;
}
```

例如：

```
{
  "timeoutMs": 300000
}
```

超过 5 分钟未完成，Runtime 应停止或转入等待 / 失败状态。

## Token 与 Cost Budget

Agent 每一步都会消耗：

- LLM input tokens
- LLM output tokens
- Tool cost
- Search / API / embedding cost

因此 Runtime 应跟踪：

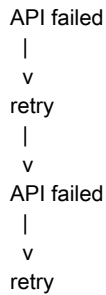
```
interface RuntimeUsage {
  inputTokens: number;
  outputTokens: number;
  toolCalls: number;
  estimatedCost: number;
}
```

当达到预算时停止。

## Tool Failure Stop

Tool 错误不能无限重试。

例如：



Runtime 需要设置：

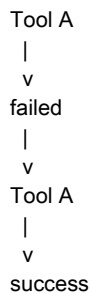
```
interface RetryPolicy {  
    maxRetries: number;  
    retryableErrors: string[];  
}
```

超过重试次数后，应产生 Error Observation 并停止或请求用户介入。

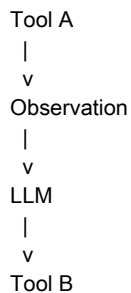
## Retry 与 Loop 的区别

Retry 和 Agent Loop 容易混淆。

Retry 是同一个 Action 的恢复：



Loop 是新的 Agent 决策：



Retry 通常由 Runtime Policy 控制。

Loop 由 LLM Decision + Runtime Control 共同控制。

## Loop Guard

Loop Guard 是 Runtime 的循环保护层。

```
interface LoopGuard {  
    check(state: RuntimeState): GuardResult;  
}
```

它可以检查：

- 是否超过最大步数
- 是否超时
- 是否超过预算
- 是否连续重复同一个 Action
- 是否连续收到相同 Observation
- 是否没有进展

例如 Duplicate Action :

```
search("abc")
search("abc")
search("abc")
```

Runtime 可以判断 Agent 陷入重复行为。

Progress Detection 更高级 :

```
第一次 Observation: 找不到文件
第二次 Observation: 还是找不到文件
第三次 Observation: 仍然找不到文件
```

如果状态没有向目标靠近，可以停止或请求用户澄清。

## Agent Executor Loop

Demo Agent 可能写成 :

```
while (true) {
  const decision = await llm();

  if (decision.final) {
    break;
  }

  await executeTool(decision.toolCall);
}
```

工业 Runtime 不会只有这么简单。

一次循环包含 :

```
Think
|
v
Decision
|
v
Validation
|
v
Permission
|
v
Execution
|
v
Observation
|
v
State Update
|
v
```

```
Loop Guard
|
v
Next Step
```

因此可以抽象为：

```
Agent Executor Loop
```

它负责控制 Agent 一轮又一轮运行。

## Agent Executor 与 Tool Executor

Tool Executor 负责一个具体 Tool 如何执行：

```
validate
|
v
permission
|
v
call api
|
v
timeout / retry
|
v
return result
```

Agent Executor Loop 负责 Agent 是否继续运行：

```
Step 1: 调用天气 Tool
Step 2: 根据天气决定是否推荐活动
Step 3: 生成回答
```

两者关系：

```
Agent Executor Loop
|
v
Tool Executor
|
v
External System
```

一句话：

Tool Executor 管单个动作，Agent Executor 管整个任务循环。

## Step Runner

工业 Runtime 常把每一次循环称为 Step。

可以抽象：

```
interface StepRunner {
  run(state: RuntimeState): Promise<StepResult>;
}
```

示例：

```
async function runStep(state: RuntimeState): Promise<StepResult> {
  const context = contextBuilder.build(state);
  const response = await llm.generate(context);
}
```

```

if (response.toolCall) {
  return executeToolStep(response.toolCall);
}

return {
  type: "final",
  answer: response.content
};
}

```

Agent Runtime 的核心不是 while，而是每个 Step 产生的 State Transition。

## Agent Loop 完整伪代码

```

async function runAgent(initialState: RuntimeState) {
  let state = initialState;

  while (true) {
    guard.check(state);

    const context =
      contextBuilder.build(state);

    const decision =
      await llm.run(context);

    if (decision.type === "final") {
      return decision.content;
    }

    if (decision.type === "tool_call") {
      const result =
        await toolExecutor.execute(
          decision.toolCall
        );

      const observation =
        resultProcessor.process(result);

      state =
        reducer(state, observation);
    }
  }
}

```

工业实现还会加入：

- Permission
- Human Approval
- Streaming Event
- Trace
- Cancellation
- Error Recovery
- Persistence

## Tool Call Queue

LLM 一次可能返回多个 Tool Call。

例如：

```
[
  {
    "name": "get_weather",
    "arguments": {
      "city": "北京"
    }
  },
  {
    "name": "get_exchange_rate",
    "arguments": {
      "from": "USD",
      "to": "CNY"
    }
  }
]
```

Runtime 可以引入 Tool Call Queue：

```
interface ToolCallQueue {
  enqueue(call: ToolCall): void;
  dequeue(): ToolCall | undefined;
}
```

流程：

```
LLM
|
v
Tool Calls
|
v
Tool Queue
|
v
Executor
```

## Sequential Tool Execution

最简单的执行方式是串行：

```
Tool A
|
v
Tool B
|
v
Tool C
```

优点：

- 简单
- 状态容易管理
- 依赖关系清楚

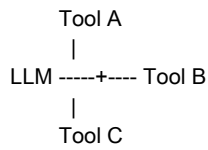
缺点：

- 慢

如果三个查询互不依赖，串行会浪费时间。

## Parallel Tool Execution

如果 Tool 之间没有依赖，可以并行：



示例：

```
await Promise.all([
  getWeather(),
  getStock(),
  getNews()
]);
```

但是并行不是默认开启，因为它有风险。

风险包括：

依赖关系

副作用

状态竞争

例如：

```
create_order()
query_order_status()
```

不能并行，因为第二个依赖第一个结果。

例如：

```
send_email()
refund_money()
```

也不应该随意并行，因为它们都有副作用。

## Tool Dependency Graph

更高级的设计不是 Queue，而是 Tool Dependency Graph。

有依赖：

```
graph TD;
  A[search_product] --> B[get_product_detail];
  B --> C[create_order];
```

无依赖：

```
graph TD;
  A[weather];
  B[news];
  C[stock];
```

可以并行。

工业 Runtime 在决定并行前，需要判断：



Dependency Graph  
+  
Side Effect  
+  
State Conflict

## OpenAI Agents SDK / Claude Code / LangGraph 映射

### OpenAI Agents SDK

从 Runtime 视角看：

```
Agent.run()
|
v
Runner Loop
|
+-- Model Turn
|
+-- Tool Execution
|
+-- Append Result
|
+-- Next Turn
|
+-- Stop Condition
```

开发者看到的是：

```
await agent.run()
```

内部是持续的：

```
LLM
|
v
Tool
|
v
Observation
|
v
LLM
```

### Claude Code

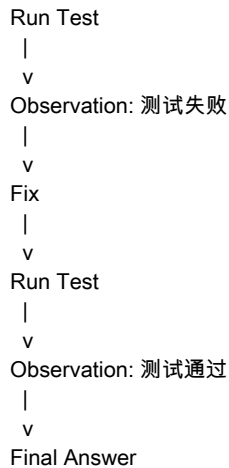
Claude Code 是 Multi Tool Loop 的典型案例。

例如用户说：

修复这个 bug

内部可能是：

```
Read File
|
v
Observation: 发现错误
|
v
Edit File
|
v
Observation: 代码修改
|
v
```



核心能力不是会编辑文件，而是能根据执行反馈调整下一步行动。

## LangGraph

LangGraph 更显式地把 Agent Loop 做成图：



它更偏显式 Workflow Graph。

Agent Loop 更偏运行时动态决策。

两者在工业系统中常常结合。

## Multi Tool Loop 与 Workflow

这是 Part H 之后最重要的补充问题。

对于流程固化能力，Multi Tool Loop 确实不如 Workflow 稳定。

原因是：

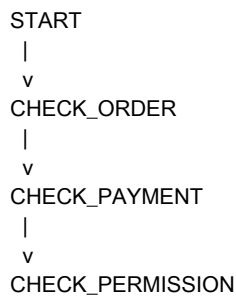
Multi Tool Loop:  
 $\text{Next Action} = f(\text{State}, \text{Context}, \text{LLM})$

下一步由当前状态、历史 Observation、Prompt 和模型能力共同决定。

Workflow 则是：

$\text{Next State} = \text{TransitionRule}(\text{CurrentState})$

例如退款流程：



```
graph TD
    A[ ] --> B[REFUND]
    B --> C[DONE]
```

下一步不是 LLM 决定，而是状态机规则决定。

所以：

Workflow 解决“确定怎么走”，Multi Tool Loop 解决“不知道下一步怎么走”。

## Workflow 管确定性

适合 Workflow 的场景：

- 支付
- 审批
- 退款
- 开户
- 合规
- 财务报销
- 企业固定业务流程

这些流程需要稳定、可控、可审计。

例如退款：

```
graph TD
    A[检查订单] --> B[检查支付]
    B --> C[检查权限]
    C --> D[审批]
    D --> E[退款]
```

企业希望 10000 次退款流程结果一致，而不是每次由模型自由发挥。

## Agent Loop 管不确定性

适合 Agent Loop 的场景：

- Coding Agent
- Research Agent
- Data Analysis Agent
- 运维排障 Agent
- 非结构化企业知识探索

这些任务的问题是路径无法提前写死。

例如修 bug：

```
search code
|
v
read file
|
v
edit
|
v
run test
|
v
observe failure
|
v
edit again
```

每个 bug 的路径都不同，Workflow 很难提前定义。

## Open-ended Capability 与超级 App 类比

Multi Tool Loop 的最大优势不是稳定，而是 Open-ended Capability。

传统超级 App 试图用 UI 聚合所有能力：

```
App
|
+-- 电商
|
+-- 外卖
|
+-- 打车
|
+-- 金融
|
+-- 内容
```

问题是用户仍然要找入口，首页越来越复杂，功能越来越深。

Agent 时代改变的是能力入口：

```
User
|
v
Natural Language Goal
|
v
Agent
|
v
Capability Selection
|
v
Business Execution
```

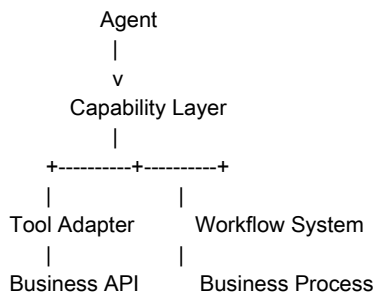
能力不再只以 UI 入口存在，也可以以 Tool 形式暴露：

```
{
  "name": "book_flight",
  "description": "预订机票"
}
```

这就是 Capability as Tool。

但 Tool 不是业务系统本身。

更合理的结构是：



例如支付不应该由 Agent 直接自由调用支付 API，而应该进入 Payment Workflow，经过风控、合规、审批、幂等后执行。

## Agent as Interface 与 Agent as Worker

学习记录中形成了一个重要区分：

Agent as Interface

和：

Agent as Worker

### Agent as Interface

Agent 主要负责入口智能化：

- 理解用户意图
- 抽取参数
- 路由到正确 Workflow
- 帮用户找到能力入口

这类场景不一定需要超大模型。

可能更适合：

Small LLM  
+  
Intent Router  
+  
Workflow Engine  
+  
Tool Ecosystem

甚至可以在手机端、本地模型或边缘设备上运行。

### Agent as Worker

Agent 负责开放任务执行：

- 探索问题
- 动态规划
- 连续调用 Tool

- 根据 Observation 修正
- 最终交付成果

这类场景需要更强模型。

例如：

- Claude Code
- Cursor
- Codex
- 数据分析 Agent
- 研究 Agent
- 自动运维 Agent

## 为什么 Coding Agent 最像 Agent

Coding Agent 是当前最像 Agent 的成熟场景，因为代码世界具备三个条件：

### 高工具密度

read\_file  
search  
edit  
terminal  
test  
git

### 高反馈密度

compile error  
test result  
runtime error  
lint error

### 高可验证性

test pass  
build success  
typecheck pass

Agent 最适合：

Action  
|  
v  
Feedback  
|  
v  
Correction

这也是为什么 Coding Agent 比很多行业“Agent”更接近 Autonomous Agent。

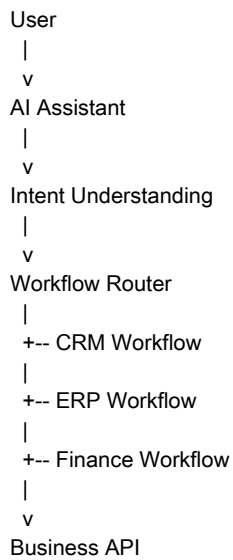
## AI Assistant + Workflow

学习记录中一个重要结论是：

目前大量企业 AI 落地并不是纯 Autonomous Agent，而是 AI Assistant + Workflow。

这不是说企业做错了，而是很多企业场景天然适合这种结构。

典型架构：



本质是：

LLM = 智能入口  
Workflow = 执行主体

例如银行客服：

User:  
我要查信用卡账单

LLM:  
intent = query\_bill

Workflow:  
身份验证 -> 查询账单系统 -> 返回结果

这里 Agent 没有复杂 Replanning，主要负责理解人话、找入口、填参数。

## 为什么企业大量选择 AI Assistant + Workflow

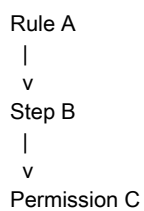
原因包括：

### 稳定

贷款、报销、退款、审批等流程不能每次由模型自由决定。

### 可控

Workflow 的每一步都可审计：



### 成本

很多企业流程不需要 GPT-5 级别大模型。

小模型 + Workflow 足够完成意图识别和参数抽取。

## 责任边界

企业业务规则、权限、安全和数据一致性仍应留在 Workflow / Service / Backend 中。

## AI Assistant + Workflow 的开源方案

学习记录中对市场开源形态做了一个工程分类。

如果只是 AI Assistant + Workflow，很多时候 LangChain.js 会偏重。

更轻量的组合是：

```
LLM API
+
Intent Router
+
Workflow Engine
+
Business API
```

常见开源方向可以分成几类：

### RAG + Assistant

适合知识问答、客服 FAQ、文档查询。

典型形态：

```
User
|
v
Retriever
|
v
LLM
|
v
Answer
```

### Workflow Automation

适合固定业务流程、审批、通知、CRM / ERP 调度。

常见工具形态包括：

- n8n
- Node-RED
- Temporal
- Windmill

### Low-code AI Workflow

适合快速搭建企业内部 AI 应用。

常见形态包括：

- Dify
- Flowise



- Langflow

## Conversational Assistant / Bot Framework

适合意图识别、槽位填充、对话流程。

常见形态包括：

- Rasa
- Botpress

## Agent Framework

适合真正需要动态规划、Tool Selection、Observation、Replanning 的场景。

常见形态包括：

- LangChain
- LangGraph
- AutoGen 类框架

因此，选择规则可以是：

| 场景 | 推荐架构 || --- | --- || 客服 FAQ | RAG + Workflow || 业务查询 | Intent Router + Workflow || 审批流程 | Workflow || ERP 操作 | Workflow || 知识助手 | RAG + Assistant || 数据分析 | Agent || Coding | Agent || 研究任务 | Agent || 自动运维 | Agent |

结论：

能用 Workflow 解决的问题，不要强行上 Agent；只有不确定的问题，才交给 Agent。

## mini-agent-runtime 设计

Part I 实现时，mini-agent-runtime 至少需要保留下面这些边界。

### Tool Schema

定义模型可见的能力契约。

### Tool Registry

管理 Runtime 可用能力。

### Tool Executor

安全执行单个 Tool。

### Result Processor

把 Tool Result / Error 转换成 Observation。

### Runtime State

保存消息、Tool Call Chain、Observation、Step、状态和 Usage。

### Agent Executor Loop

控制多轮执行。

### Loop Guard

控制最大步数、超时、预算、重复动作和失败次数。

## Planner 接口

为了未来支持 Agent + Workflow，可以预留：

```
interface Planner {  
  decide(state: RuntimeState): Promise<NextAction>;  
}
```

未来可以有：

```
LLMPlanner  
WorkflowPlanner  
HybridPlanner
```

这样 mini-agent-runtime 不只支持纯 Agent Loop，也能逐步过渡到 Workflow + Agent 的混合架构。

## 工业术语映射

| 本项目概念 | 工业术语 || --- | --- || Multi Tool Loop | Agent Loop / ReAct Loop || Agent Executor Loop | Runner / Agent Runner / Execution Loop || ToolCallRecord | Tool Call Trace / Action Record || Observation | Tool Observation / Runtime Feedback || Runtime State | Agent State / Workflow State || Loop Guard | Loop Control / Guardrail / Runtime Policy || Stop Condition | Termination Condition / Max Turns / Max Steps || Tool Call Queue | Tool Queue / Action Queue || Tool Dependency Graph | Execution DAG / Tool DAG || Workflow Constraint | Guarded Workflow / Agentic Workflow || AI Assistant + Workflow | Tool-using Assistant / Intent Router + Workflow |

## 面试视角

Q1：为什么 Agent 需要 Multi Tool Loop？

答：

单次 Tool Calling 只能完成一次动作，而真实任务通常需要根据执行结果动态决定下一步。因此 Agent Runtime 需要维护 Think-Act-Observe 循环，把 Tool Result 转换成 Observation，更新 Runtime State，再进入下一轮决策。

Q2：Tool Calling 为什么不等于 Agent？

答：

Tool Calling 只提供单次动作能力，而 Agent 需要根据执行结果动态调整下一步行动，因此必须具备 Loop、State Management、Observation Feedback 和 Control Policy。

Q3：为什么不能把停止条件交给 LLM？

答：

LLM 擅长语义决策，但不适合负责资源、安全和成本控制。工业 Runtime 必须通过 max steps、timeout、token budget、cost budget、retry policy 和 duplicate detection 管理执行边界。

Q4：Tool Executor 和 Agent Executor 有什么区别？

答：

Tool Executor 负责一个具体 Tool 的生命周期管理，包括参数校验、权限、执行、重试和错误处理；Agent Executor 负责整个 Agent Loop，包括 LLM 调用、状态推进、循环控制和停止判断。

## Q5 : Agent Loop 和 Workflow 有什么区别？

答：

Workflow 的执行路径通常由开发者提前定义，而 Agent Loop 的下一步行动由 LLM 根据当前 State 和 Observation 动态决定。工业系统通常结合两者，用 Workflow 保证确定性流程，用 Agent Loop 处理开放任务。

## Q6 : 为什么很多企业 Agent 更像 AI Assistant + Workflow？

答：

因为大量企业业务高度结构化，真正需要的是稳定、可审计、低成本流程执行。LLM 在其中主要承担意图识别、参数抽取和入口智能化，执行主体仍是 Workflow 和业务系统。

## 下一节学习计划

下一节进入：

Day05 Part I : Mini Tool Runtime Implementation

Part I 会把 Day05 Part A-H 的设计组合成一个最小可运行 Runtime：

```
Tool Schema
+
Tool Registry
+
Tool Executor
+
Permission Layer
+
Observation
+
Runtime State
+
Multi Tool Loop
|
v
Mini Agent Runtime
```

主要内容：

Runtime 基础骨架

Tool 系统实现

Agent Loop 实现

Observation Pipeline

Loop Guard

完整 Demo

## 写书 TODO

### Chapter : Agent Runtime Loop

需要补充以下章节素材。

Agent Loop 为什么是 Agent Runtime 核心

Tool Calling 让模型拥有行动能力，Loop 让模型拥有持续完成目标的能力

Agent Loop 不等于 while(true)

ToolCall 与 Observation 必须分离

Runtime Control 与 LLM Decision 必须分离

Multi Tool Loop 是动态 Workflow，但不是 Workflow 的替代品

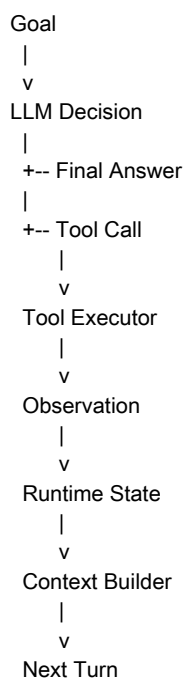
Loop Guard 是生产级 Agent 的安全、成本和可靠性边界

并行 Tool 是 Runtime 调度问题，不只是 Promise.all

企业 AI 落地中 AI Assistant + Workflow 与 Autonomous Agent 的边界

## 写书素材

### 素材 1：Agent Loop 生命周期图

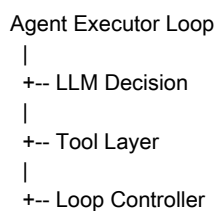


### 素材 2：ToolCall 与 Observation 分离

ToolCall:  
我要执行什么

Observation:  
世界发生了什么

### 素材 3：Runtime 三层结构



### 素材 4：Agent 与 Workflow 的边界

Workflow:  
确定怎么走

Agent Loop:  
不知道下一步怎么走时动态探索

## 素材 5 : AI Assistant + Workflow

LLM = 智能入口  
Workflow = 执行主体  
Business System = 权威边界

## 本 Part 核心认知升级

### 升级 1

从：

Agent 会调用 Tool

升级为：

Agent 是一个基于反馈持续执行的循环

### 升级 2

从：

多 Tool = 多函数调用

升级为：

Multi Tool = Tool Call Chain + Observation + State Transition

### 升级 3

从：

LLM 控制 Agent

升级为：

LLM 负责决策，Runtime 负责治理

### 升级 4

从：

Agent Loop 是代码循环

升级为：

Agent Loop 是事件驱动状态机

### 升级 5

从：

Workflow 和 Agent 二选一

升级为：

Workflow 管确定性，Agent Loop 管开放性

### 升级 6

从：

企业 Agent 都应该做成 Autonomous Agent

升级为：

## 工业级实现

生产级 Runtime 必须拥有 Loop Governance :

- max steps
- timeout
- token budget
- cost budget
- retry policy
- duplicate action detection
- progress detection
- cancellation
- trace and audit

生产级 Runtime 通常事件化记录 :

AgentStarted  
ModelCalled  
ToolCalled  
ToolCompleted  
ObservationCreated  
StateUpdated  
AgentFinished

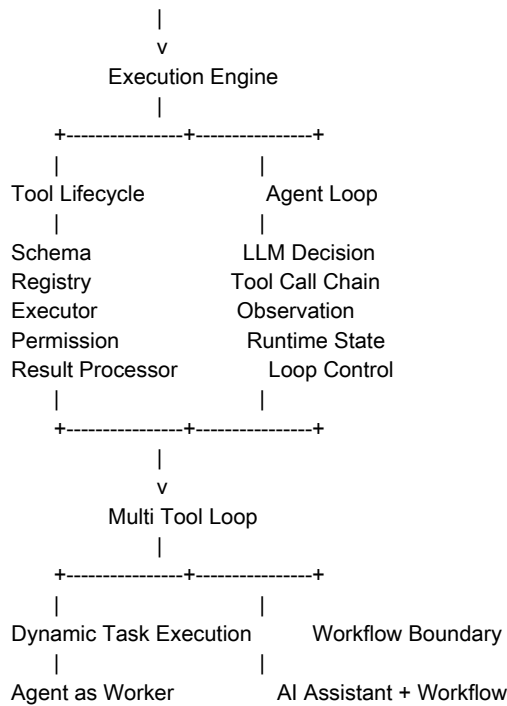
这些事件用于 :

- Debug
- Trace
- Replay
- Evaluation
- Audit
- Recovery

Parallel Tool Execution 必须考虑 :

- Tool dependency
- Side effect
- State conflict
- Permission
- Idempotency
- Result ordering

## 知识地图



## 思考题

为什么 Tool Calling 不等于 Agent ？

为什么 Tool Result 不能直接等于下一轮 Prompt ，而需要 Observation ？

为什么 Runtime State 和 Context 必须分离 ？

如果 Agent 连续 10 次调用同一个 Tool 都失败，应该由谁决定停止 ？

查询天气和查询新闻为什么可以并行？创建订单和查询订单状态为什么不能并行？

Agent Loop 和 Workflow 的边界在哪里？

什么场景适合 AI Assistant + Workflow ，而不是 Autonomous Agent ？

为什么 Coding Agent 是当前最像 Agent 的成熟场景？

mini-agent-runtime 中 Planner 接口为什么有助于未来支持 WorkflowPlanner / HybridPlanner ？

## 前置问题回收

Q1：Tool Result 到 Observation 的关系是什么？

Tool Result 是执行输出，Observation 是 Runtime 可理解、可存储、可投影、可审计的反馈事件。Observation 是下一轮决策输入。

Q2：人工审批暂停是否导致普通请求超时？

不应该。工业 Runtime 会保存 Pending State、Resume Token 和 Waiting Status ，等待外部 Human Decision 后恢复。

Q3：Tool 与 MCP 的边界是什么？

Tool 是 Runtime 内部能力抽象，MCP 是外部能力协议。无论 Tool 来源于本地函数、HTTP、Plugin 还是 MCP，最终都应该进入统一 Tool Call -> Executor -> Observation -> Loop。

Q4：流程固化能力是否更适合 Workflow？

是。对稳定、可审计、强规则的业务流程，Workflow 通常比 Multi Tool Loop 更合适。Agent Loop 更适合开放任务和不确定路径。

Q5：很多企业所谓 Agent 是否其实是 AI Assistant + Workflow？

很多是。它们主要解决能力发现、意图识别和参数抽取，执行主体仍是 Workflow 和业务系统。这是合理路线，不是错误路线。

## 参考

- ChatGPT 分享记录：<https://chatgpt.com/share/6a72f6c8-2228-83e8-ad10-0e94c42d3e6e>

- 本地源记录：source/day05-part-h-chatgpt-share-source.md